



AI 辅助科研方法：工具链与设计思路

读得更宽 · 做得更快 · 流程可编排

摘要

本文档记录一套 AI 辅助科研方法的设计思路，以及配套演示文稿的内容总结。

核心思路：科研中的三个常见瓶颈——**阅读上下文有限**（读不宽）、**执行效率不足**（做得慢）、**读与做断裂**（缺少流程编排）——可以通过三个工具组合成闭环来应对。

Vibero 负责扩展阅读边界，OpenCode 负责提升执行力，Research Assistant 负责将整个流程编排为可重复的管线。

设计上的关键点：(1) 用深度报道为 AI 建立语境后再做概念联想，避免概率式随机猜测；(2) 范文放在初稿完成后做论证对照，不进入主生成流程；(3) 评审发现的问题按路由表定向修复，不全文重写。

一 问题背景

日常科研中，有三个环节容易卡住：



上下文上限

阅读带宽有限，文献却不断积累。容易遗漏关键脉络，或把



执行效率

从想法到可运行产物（脚本、图表、初稿）之间有大量重复



流程断裂

阅读和写作各自为战。读了 50 篇论文，但没有系统把阅

 注意力平均分配在次要内容上。	性工作，消耗本可用于思考的注意力。	读成果组织成论证结构。
--	-------------------	-------------

这三个问题指向同一个方向：科研中有大量工序可以自动化，而自动化之后释放出来的注意力，应该用在判断上——选什么理论框架、论点是否成立、结论是否过度推广。关键是区分什么可以交给工具，什么必须留在人手里。

二 工具链：三条腿拼成闭环

演示文稿（deck）的叙事顺序就是工具链的展开逻辑：先诊断瓶颈，再逐一展开对应工具，最后收束到方法论。

2.1 第一条腿：读 – Vibero

解决什么问题：上下文上限。传统线性阅读（从摘要读到结论）默认每篇论文都值得同等的注意力，但实际上大部分论文仅 20% 的内容与研究者的需求直接相关。

Vibero 的切入方式：

- 非线性阅读 —— 把 PDF 变成可交互的 AI 画布，先让 AI 解析论文结构树，再决定注意力投在哪里
- 核心点定位 —— "先见森林，再见树木"，先抓整体论点和论证骨架，再选关键段落细读
- 画布与对话联动 —— 点选论文段落直接追问 AI，回答可拖回画布作为笔记

本质是把"非线性注意力"这个阅读策略做成了工具——不是替代精读，是让研究者更高效地分配有限的阅读注意力。

2.2 第二条腿：做 – OpenCode



解决什么问题：执行效率。数据清洗、统计脚本、格式调整、报告骨架——这些工作消耗注意力但不消耗判断力，适合交给 Agent 完成。

OpenCode 的切入方式：

- 项目上下文 —— 能理解整个项目目录结构，不需要每次都粘贴背景信息
- 多步执行 —— 能把复杂任务拆解为 Todo 并逐个执行（读数据 → 清洗 → 分析 → 可视化 → 嵌入报告）
- 开源、可接多种模型 —— 不绑定单一厂商

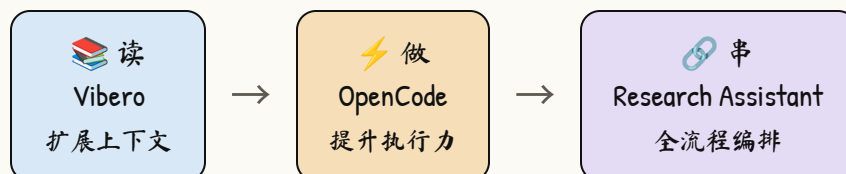
与聊天式 AI 的区别在于：OpenCode 不只是回答问题，而是在项目环境中实际执行任务——改文件、跑脚本、生成产出物。

2.3 第三条腿：串 — Research Assistant

解决什么问题：流程断裂。Vibero 和 OpenCode 各自解决了读和做的问题，但两者之间缺少衔接——阅读成果如何进入论证结构？执行结果如何嵌入论文章节？

Research Assistant 的切入方式：

- 定义完整的 6 阶段研究流程（问题锚定 → 文献搜索 → 实证设计 → 数据图表 → 初稿 → 评审 → 修订 → 交付）
- 每阶段设置质量关卡，不通过不进入下一阶段
- 评审发现问题后按弱点路由表定向修复，不全文重写
- Phase 3 · 4（实证设计、数据图表）对纯理论论文自动跳过



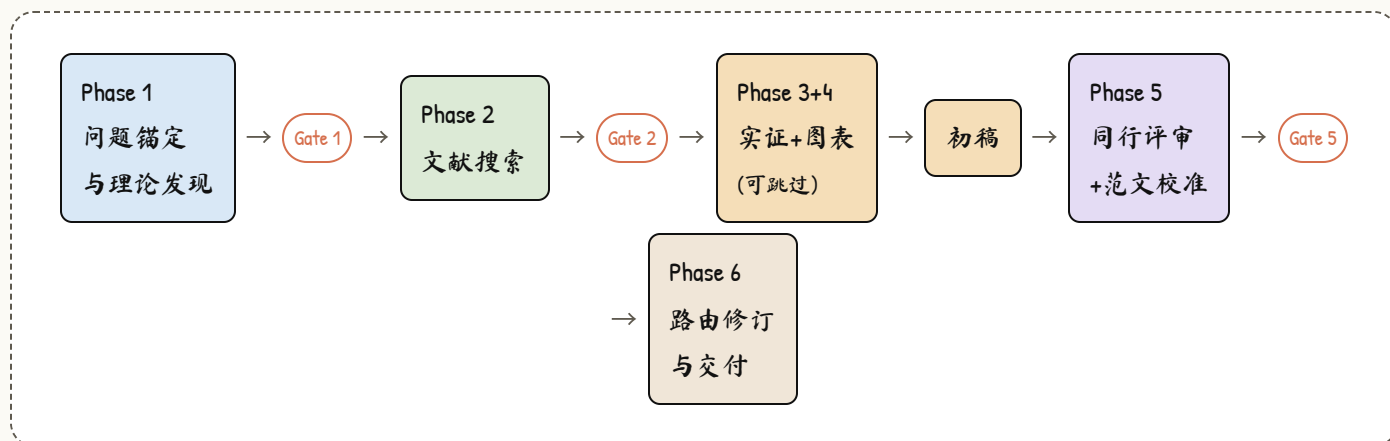
= 科研闭环





Research Assistant 设计要点

3.1 管线结构



3.2 语境锚点法

Phase 1 要解决的问题：对 AI 说"我对 X 现象感兴趣，帮我找理论"，AI 只能从关键词出发做概率联想。因为它对 X 没有画面感——不知道具体是谁、在什么条件下、面临什么约束。

方法：先给 AI 喂深度报道（或 CS 的 survey/SoK），让它从具体细节中建立脉络，再从脉络联想学术概念。社科用深度报道/案例，CS 用技术综述——锚点材料不同，逻辑一致。

联想出的概念再丢进 Google Scholar 按引用量验证，选影响力大的概念构建理论地图，避免过早收敛到小众概念。

3.3 范文后置校准

传统做法是把范文放在流程前端当模板——AI 按范文结构填空。两个问题：范文的论证结构会替代研究者自己的论证生长路径；范文领域不同时，机械套用会压制跨学科创新。

这里的做法：范文不进主生成流程。初稿完成后，找同类型顶刊范文对照论证结构——范文是校准工具，不是思维模具。

3.4 质量关卡与弱点路由



关卡	位置	检查项
Gate 1	Phase 1→2	深度报道已找到、语境提取完整、概念联想有细节支撑
Gate 2	Phase 2→初稿	数据库已调用、引用来源可验证、覆盖核心文献
Gate 5	Phase 5→6	所有 Gate 通过、评审达标、范文对照无结构性缺陷

发现问题时不全文重写，按路由表定向分发：

问题类型	路由	修复方式
概念联想缺乏语境支撑	Phase 1	重新搜索深度报道
引用覆盖不足	Phase 2	定向补充文献
章节结构混乱	Phase 6	重组章节、增加过渡
论证薄弱	Phase 6	增加批判性分析段落
论断过强	Phase 6	降级为 hedge language

四

演示文稿内容

配套文稿共 9 页，HTML deck 翻页形式，Ian Handdrawn 手绘风格。叙事顺序对应方法论的展开逻辑。

01 封面 — How to do Research

封面

主标题"科研方法分享"，副标题"扩展上下文 · 提升执行力 · 流程可编排"。中央手绘双轴简图（纵轴上下文、横轴执行力），底部标签列出三个工具。

02 科研闭环 — 两条瓶颈

哲学 ★

提出核心诊断：科研受限于"上下文上限 × 执行效率"。引入"非线性注意力"概念。后续 Vibero 和 OpenCode 分别落在这两条腿上。





03 Vibero — 氛围阅读

产品简介 + 品牌页截图。四个能力标签：非线性阅读 / 核心点定位 / 画布对话 / 论文代码。

04 Vibero Demo — 非线性阅读

Demo

以 Transformer 论文为例。流程：打开论文 → VIBE 解析结构树 → 点选段落追问 → 笔记留在画布。主张"先见森林，再见树木"。

05 OpenCode — 开源 AI 编程 Agent

OpenCode

左栏要点（读代码库 / 改文件 / 多步执行 / 项目上下文），右栏官网截图。强调对科研的价值：想法转化为可运行产物。

06 OpenCode Demo

Demo

舞台感布局。中央 LIVE 徽章 + 演示区，左侧三步编号（打开网站 → 安装 → 跑任务）。强调人在把关、Agent 在执行。

07 Research Assistant — 串成闭环

RA ★

左栏：读（Vibero）→ 做（OpenCode）→ 串（Research Assistant）。底部注：6 阶段管线带质量门禁。三条腿各解决一个瓶颈，RA 把它们串起来。

08 收束 — 方法论，不是推销

收束

三格面板呼应开篇。收束句：扩展上下文 + 提升执行；判断在人，工具负责读与跑；工具可换，哲学不变。以 Q&A 收尾。

09 彩蛋 — Auto Research

彩蛋

展示进阶方向：全流程自动化，人留在问题发现与质量判断上。放在彩蛋页——日常闭环（读 → 做 → 串）才是听众能直接带走的东西。



总结

整个项目的底层逻辑可以归纳为几条：



- 
- 1. 先诊断，再找工具。** 三个瓶颈（上下文上限、执行效率、流程断裂）是问题驱动的，工具是为了解决具体瓶颈而选，不是先有工具再拼凑理论。
 - 2. 三条腿缺一不可。** 只读不做——想法停留在脑海。只做不读——执行再快也缺乏文献根基。读和做都有了但没有编排——流程是断裂的。三者拼在一起才形成闭环。
 - 3. 区分什么可以自动化、什么不可以。** 工序可自动化（检索、格式、检查清单），判断不可自动化（问题选择、理论取舍、论点裁决）。质量关卡设在每个"判断必须介入"的节点。
 - 4. 工具可换，哲学不变。** Vibero 可能被更好的阅读器替代，OpenCode 可能被其他 Agent 替代。但底层方向——扩展上下文 + 提升执行 + 自动化工序——是跨工具的。

读得更宽 · 做得更快 · 流程可编排
想法在人 · 工序可自动化

 参考来源

概念	来源
迭代环 + 弱点路由表 + 质量关卡	陈德里 DeliAutoResearch (2026)
语境锚点法	孙宇凡 "AI 赋能理论框架三步法"
范文后置校准	孙宇凡 "AI 仿写顶刊四步法"
多 Agent 交叉验证	GPT Researcher + Academic Research Skills
Ralph Loop 定向迭代	Claude Code 官方插件
四索引交叉核验	Academic Research Skills v3.11.0
CS/STEM 锚点扩展	社区贡献



